

Project 2

Gabor Stefan-loan_group_407

Exercise 1:

First I wrote my own rules.

Rules:

1. If you write rhymes, you make poetry.
2. If you make music, and poetry, you are a singer.
3. if you are a singer and have 100.000 fans, you are successful singer.

And my own questions:

1. Are you write rhymes? (answer is yes or no)
2. Are you make music? (answer is yes or no)
3. How many fans do you have? (answer is a number)

The program give a quiz to the user, and based on user's answers, the program would decide, if the user is a successful singer or not.

Here, if the user answers 'yes' to the first two questions, and type a number greater than 100000 to the third one, then the program will output 'yes' (means that the user is a great singer), otherwise will be 'no'

The rules as Horn clauses in CNF are:

1. $[n(\text{rhymes}), \text{poetry}]$
2. $[n(\text{music}), n(\text{poetry}), \text{singer}]$
3. $[n(\text{singer}), n(\text{fans}), \text{successful_singer}]$,

where $n(A)$ is the atom A negated.

To make this program, both forward and backward chaining were implemented.

Some Important predicates are:

`Start_quiz()`: - this starts the quiz.

`Read_rules(X)` :- reads Horn clauses in CNF from a file, X take the list of rules.

`ask(X)` :- ask the user questions, and then all answers are saved in X.

`backward_chaining(KB,Goal)` :- apply the backward chaining algorithm itself.

subtract_helper :- this is used in backward_chaining predicate to resolve two items.

Forward_chaining(KB,Solved,Goal) :- this apply the forward chaining algorithm itself.

Get_new_members(KB,Solved, List, new_members) :- this find new atoms which can be entailed i.e. all it's premises are solved.

The prolog code for exercise 1:

```
negate(n(A),A):-!.
negate(A,n(A)):-!.

processA1(stop,_) :- halt.
processA1(Usr_ans, A1) :- Usr_ans == yes, append([], [rhymes], A1).
processA1(Usr_ans, A1) :- Usr_ans == no, append([], [n(rhymes)], A1).
processA1(Usr_ans, A1) :- writeln("please answer only with 'yes' or 'no'"),
ask1(A1).

processA2(Usr_ans, A2) :- Usr_ans == yes, append([], [music], A2).
processA2(Usr_ans, A2) :- Usr_ans == no, append([], [n(music)], A2).
processA2(Usr_ans, A2) :- writeln("please answer only with 'yes' or 'no'"),
ask2(A2).

processA3(Usr_ans, A3) :- not(number(Usr_ans)), writeln("please answer only with
a number e.g. '43'"), ask3(A3).
processA3(Usr_ans, A3) :- Usr_ans >= 100000, append([], [fans], A3).
processA3(Usr_ans, A3) :- Usr_ans < 100000, append([], [n(fans)], A3).

ask1(Ans1) :- write("are you write rhymes?(yes/no)"), read(Usr_ans1),
processA1(Usr_ans1, Ans1).
ask2(Ans2) :- write("are you make music?(yes/no)"), read(Usr_ans2),
processA2(Usr_ans2, Ans2).
ask3(Ans3) :- write("how many fans do you have?(type a number)"), read(Usr_ans3),
processA3(Usr_ans3, Ans3).

ask([Ans1, Ans2, Ans3]) :- ask1(Ans1), ask2(Ans2), ask3(Ans3).

read_rules(Rules) :-
    open('Rules.txt', read, Str),
    read(Str, Rules),
    close(Str).
```

```

check_final_response(stop) :- true,!.
check_final_response(_) :- false,!.

final_response() :-
    writeln("if you want to reload the quiz type 'stop', otherwise type any key
and press enter"),
    read(Usr_response),
    Usr_response = stop.
final_response() :- false.

start_quiz() :-
    read_rules(Rules),
    repeat,
    ask(RList),
    append(Rules,RList, KB),
    backward_chaining(KB,[n(successful_singer)]),
    forward_chaining(KB,[],successful_singer),
    writeln('Continue? press stop if you dont want'),
    final_response().

merge_lists_no_duplicates(LIST1, LIST2, RESULT):-
    append(LIST1,LIST2,R), list_to_set(R, RESULT).

subtract_helper(A,B,[NA,NB], Resolvent):-
    subtract(A,[NA],A_SUBS),
    subtract(B,[NB],B_SUBS),
    merge_lists_no_duplicates(A_SUBS,B_SUBS,NEW_LIST),
    sort(NEW_LIST,N_L_S),
    append([],N_L_S, Resolvent).

backward_chaining(KB, []) :- writeln("backward chaining result : yes").

backward_chaining(KB, Goal) :-
    member(Goal_atom, Goal),
    member(C, KB),
    negate(Goal_atom, NG_atom),
    member(NG_atom, C),
    subtract_helper(Goal,C,[Goal_atom,NG_atom], RES),
    backward_chaining(KB,RES).

backward_chaining(KB,_) :- writeln("backward chaining result : no").

```

```

get_positive_atom([],P) :-!.
get_positive_atom([A|B],A) :-
    not(A = n(_)),!.
get_positive_atom([A|B],P) :- get_positive_atom(B,P),!.

turn_positive([],[]) :-!.
turn_positive([n(A)|B],[A|T]) :- turn_positive(B,T).

check_solved_new([],_) :- true.
check_solved_new([A|B],Solved) :-
    member(A,Solved),
    check_solved_new(B,Solved).
check_solved_new(_,_) :- false.

get_new_members([], Solved,List, List) :-!.
get_new_members([C|B], Solved, List, New_Members):-
    get_positive_atom(C, Pos_at),
    subtract(C, [Pos_at], Neg_C),
    turn_positive(Neg_C, Pos_C),
    check_solved_new(Pos_C, Solved),
    not(member(Pos_at, Solved)),
    merge_lists_no_duplicates(List, [Pos_at], New_List),
    get_new_members(B,Solved, New_List,New_Members),!.

get_new_members([C|B], Solved, List, New_Members) :- get_new_members(B, Solved,
List, New_Members).

forward_chaining(KB, Solved, Goal) :- member(Goal,Solved), writeln("forward
chaining result : yes"),!.
forward_chaining(KB,Solved, Goal) :-
    get_new_members(KB,Solved,[],New_Members),
    not(New_Members = []),
    append(Solved,New_Members, Solved_up),
    forward_chaining(KB,Solved_up, Goal).
forward_chaining(_,_,_) :- writeln("forward chaining result : no").

```

an example of program usage, where the answer was yes:

```
?- start_quiz().
are you write rhymes?(yes/no)yes.
are you make music?(yes/no)|: yes.
how many fans do you have?(type a number)|: 10000000.
backward chaining result : yes
forward chaining result : yes
Continue?
if you want to stop the quiz type 'stop', otherwise type any key and p
ress enter
|: █
```

Exercise 2.

Rules:

If the political regime is totalitarian, or the average income is low, then the quality of life is lower.

If the political regime is democratic, or the average income is medium then the quality of life is medium.

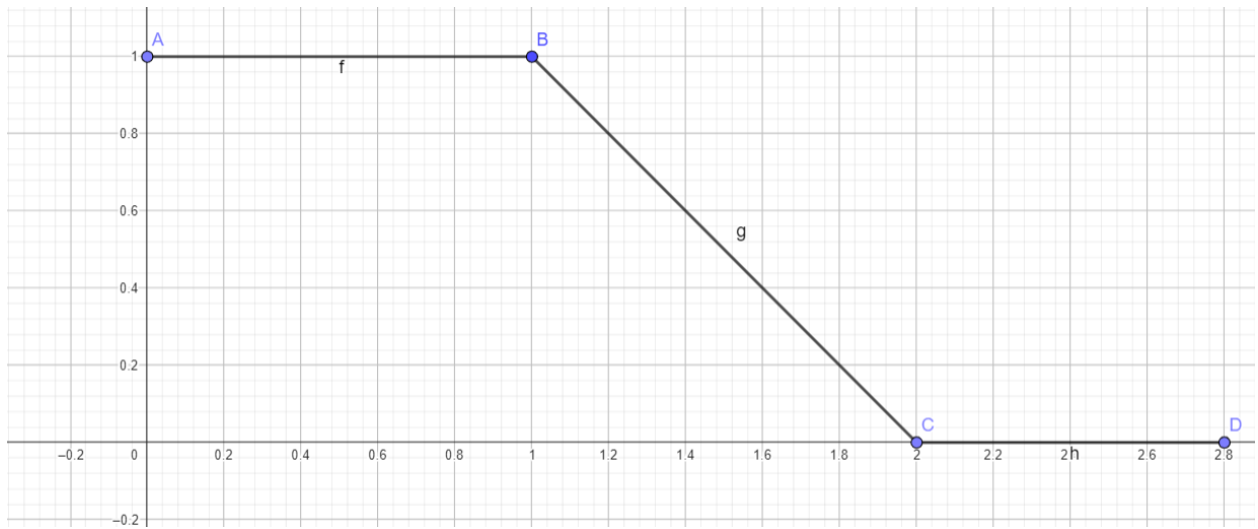
If the political regime is altruist, or the average income is high, then the quality of life is high.

The idea is that based on how the oppressive regime is, and the average income, someone can use this program to find the life quality on every country.

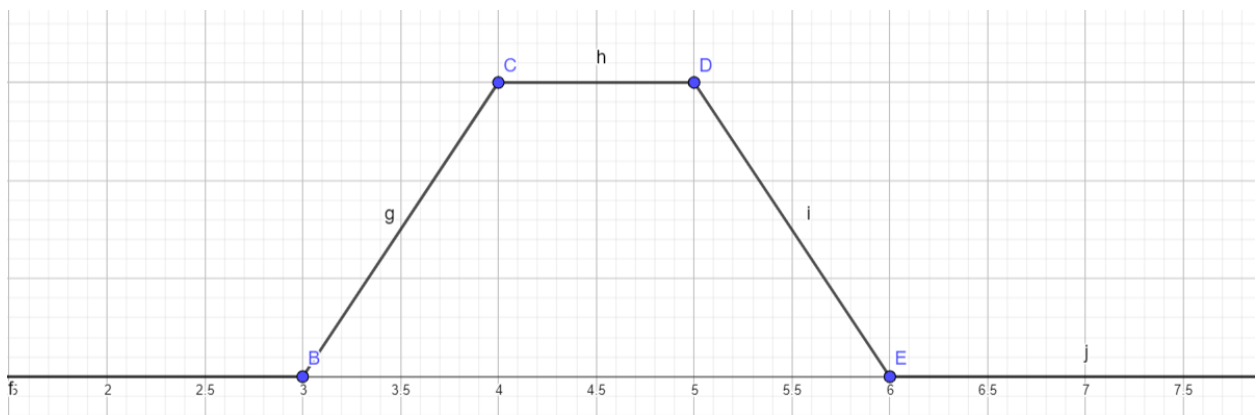
In the txt. File I wrote these rules as follows:

```
[[or(political_regime/totalitarian, income/lower), life_quality/lower],
[or(political_regime/democratic, income/average), life_quality/medium],
[or(political_regime/altruist, income/high), life_quality/high]].
```

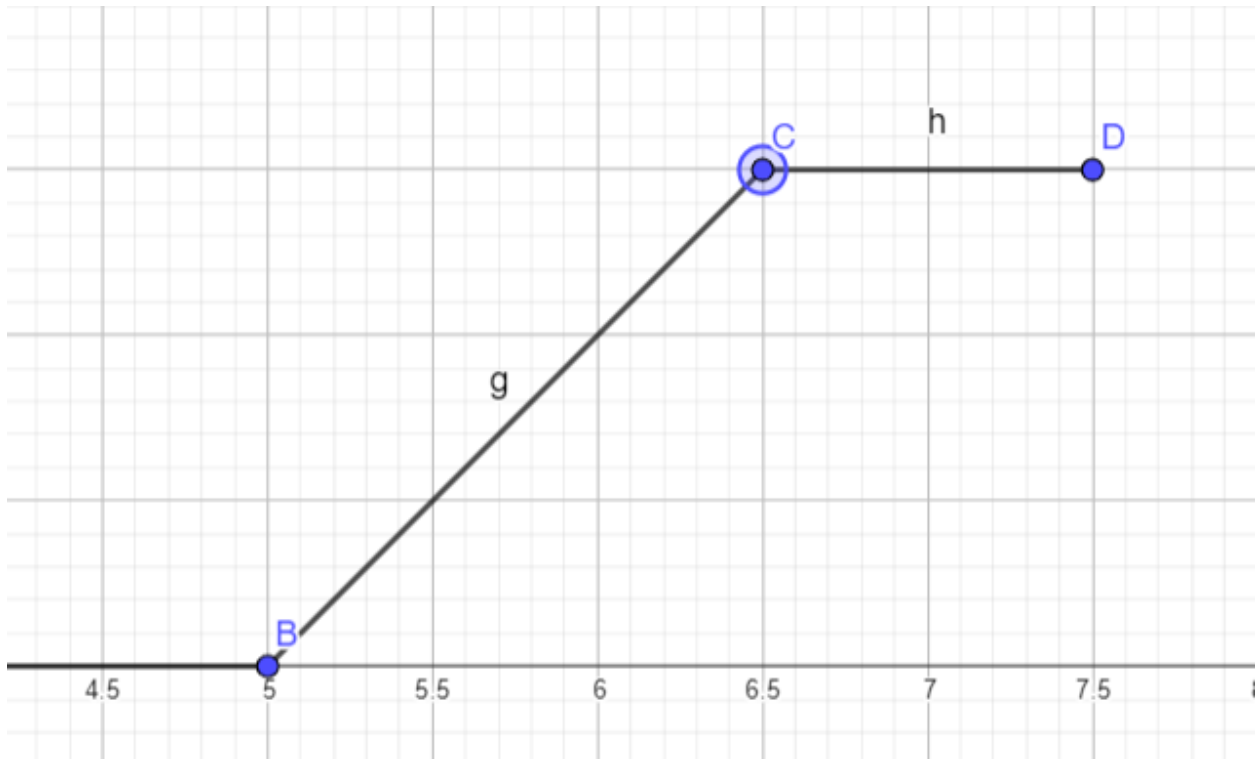
And I designed the degree curves as follows.



Oppression on altruist politic regime.



Oppression in a totalitarian politic regime.



Oppression in a totalitarian politic regime.

The code for the exercise 2:

```
read_rules(Rules) :-
    open('Ex 2 Rules text.txt', read, Str),
    read(Str, Rules),
    close(Str).

processA1(X, X) :-
    number(X),
    10 >= X,
    X >= 0.

processA1(X, A1) :-
    writeln("please type only a number on a scale from 1 to 10"),
    ask1(_).

processA2(X, X) :-
    number(X),
    10 >= X,
    X >= 0.
```

```
processA2(X,A1) :-  
    writeln("please type only a number on a scale from 1 to 10"),  
    ask1(_).  
  
ask1(Ans1) :- writeln("how oppressive si the political regime, you live on a scale  
from 1 to 10"), nl, read(Usr_ans1), processA1(Usr_ans1 ,Ans1).  
ask2(Ans2) :- writeln("how big is the average income, on a scale from 1 to 10?"),  
nl, read(Usr_ans2), processA2(Usr_ans2,Ans2).  
  
ask([Ans1,Ans2]) :- ask1(Ans1), ask2(Ans2).  
  
start_quiz(Rules,Ans) :- read_rules(Rules),ask(Ans).
```